Transcrição

**Olá, pessoal, boas-vindas de volta! **

Neste vídeo, vamos **explorar a teoria** por trás do que já abordamos até agora. Para alguns conceitos adicionais, vamos abrir o terminal. Nele, digitamos `php -a` para **abrir um terminal interativo** que nos permite executar código PHP e observar os resultados.

```
php -a
```

COPIAR CÓDIGO

Como retorno, obtemos:

```
Interactive shell
```

Primeiro, digitamos um `require`. Entre aspas simples, por ser uma *string*, digitamos `'src/Modelo/Filme.php'`.

Fechamos as aspas e adicionamos um ponto e vírgula.

```
require 'src/Modelo/Filme.php';
```

COPIAR CÓDIGO

Com isso, adicionamos esse arquivo. Agora, podemos **criar um novo filme**.

Criando um novo filme

Definimos uma variável `$filme` como `new Filme()`.

```
$filme = new Filme();
```

COPIAR CÓDIGO

Em seguida, **atribuímos um nome ao filme** com `$filme->nome`, colocando o nome entre aspas. Por enquanto, vamos atribuir apenas o `'Nome do filme'`.

```
$filme->nome = 'Nome do filme';
```

COPIAR CÓDIGO

Vamos fazer um `var_dump` passando esse filme como parâmetro para verificar seu conteúdo.

```
var_dump = $filme;
```

COPIAR CÓDIGO

Como retorno, obtemos o seguinte **objeto**:

```
object(Filme)#1 (1) {
    ["nome"]=>
    string(13) "Nome do filme"
    ["anoLancamento"]=>
    uninitialized(int)
    ["genero"]=>
    uninitialized(string)
    ["nota"]=>
    uninitialized(float)
}
```

COPIAR CÓDIGO

Temos um `objeto` do tipo `Filme`, mais adiante vamos entender melhor sobre isso. Há um número `#1`, seja lá o que isso for, e um `nome` definido como o `'Nome do filme'`.

Logo após, temos o **ano de lançamento**, **gênero** e **nota** que não estão inicializados (`uninitialized`). Observem que **não atribuímos nenhum valor a essas variáveis**, por padrão, estão vazias.

Ao tentarmos acessar o valor de `echo %filme->anoLancamento`, receberemos um **erro** indicando que estamos tentando acessar um valor que **ainda não foi inicializado**.

```
echo %filme->anoLancamento;
```

COPIAR CÓDIGO

Como retorno, obtemos:

> Warning: Uncaught Error: Typed property Filme::$anoLancamento must not be accessed before initialization in php shell code:1
>
> Stack trace:
>
> #0 (main)

```
thrown in php shell code on line 1
```

Isso impede que o código continue executando e resulta em um erro.

Agora, passando para a parte teórica, notem que ao fazermos um `var_dump`, a palavra `object` (objeto) aparece.

```
object(Filme)#1 (1)
```

Isso significa que a classe que criamos define um novo tipo, assim como um tipo inteiro ou uma *string*. Portanto, criamos um **tipo complexo**, composto por outros tipos, que denominamos como tipo `Filme`. Essa é a **definição de uma classe**.

Quando criamos uma variável do tipo de uma classe, como uma variável do tipo `Filme`, **estamos criando um objeto**. Outra palavra para criar um objeto é **instanciar**. Portanto, estamos instanciando um objeto do tipo `Filme`, ou seja, a variável `Filme` é uma instância do tipo `Filme`.

Embora pareça haver muitos termos para entender, tudo isso se torna mais claro à medida que nos adaptamos a esse novo paradigma de programação, que utiliza classes e objetos. Basicamente, a classe é

como a forma do bolo, os ingredientes e a receita; é como você cria um bolo.

O objeto é o bolo em si. A partir da receita, da forma e dos ingredientes, você gera o bolo. Depois de assado, você tem o bolo pronto. A partir da mesma forma e receita, é possível criar vários bolos. Da mesma maneira, **a partir da mesma classe**, podemos criar diferentes objetos. Por exemplo, podemos criar a variável `filme2`, que será um novo filme.

```
$filme2 = new Filme();
```

COPIAR CÓDIGO

Com isso, entendemos que é possível ter **vários objetos da mesma classe**, ou seja, **várias instâncias da mesma classe**. Em outras palavras, podemos criar múltiplos objetos a partir de uma única classe.

Compreendido o que é uma classe e o que é um objeto, podemos avançar para um segundo ponto importante: **a criação da variável** `filme2`, que é uma nova instância da classe `Filme`.

Se definirmos `anoLancamento` de `filme2` como `2022`:

```
$filme2->anoLancamento = 2022;
```

COPIAR CÓDIGO

E na sequência, realizarmos um `var_dump` em `filme2`, obtemos o seguinte retorno:

```
var_dump($filme2);
```

COPIAR CÓDIGO

```
object(Filme)#2 (1) {
["nome"]=>
uninitialized(string)
["anoLancamento"]=>
int(2022)
["genero"]=>
uninitialized(string)
["nota"]=>
uninitialized(float)
}
```

COPIAR CÓDIGO

Observamos que todas as variáveis, exceto `anoLancamento`, ainda não foram inicializadas, com anoLancamento configurado como 2022. Por outro lado, se realizarmos um `var_dump` em `filme`:

```
var_dump($filme);
```

COPIAR CÓDIGO

Obtemos o seguinte retorno:

```
object(Filme)#1 (1) {
["nome"]=>
string(13) "Nome do filme"
["anoLancamento"]=>
uninitialized(int)
["genero"]=>
uninitialized(string)
["nota"]=>
uninitialized(float)
```

}

Observamos que a primeira variável que criamos, apenas o `nome` estará definido. Até aqui, tudo está claro e sem novidades.

Agora, o que vamos fazer é atribuir a `filme` o valor de `filme2`, ou seja, a **variável `filme` passará a possuir os mesmos valores que `filme2`.**

```
$filme = $filme2;
```

COPIAR CÓDIGO

Quando realizarmos um `var_dump` em `filme`, veremos que o `nome` estará **não inicializado** e o **ano de lançamento** será **2022**. Isso não é inesperado.

```
var_dump($filme);
```

COPIAR CÓDIGO

Como retorno, obtemos:

```
object(Filme)#2 (1) {
["nome"]=>
uninitialized(string)
```

```
["anoLancamento"]=>
int(2022)
["genero"]=>
uninitialized(string)
["nota"]=>
uninitialized(float)
}
```

A partir de agora, o valor da variável `filme` é o mesmo que o da variável `filme2`, então **todos os valores serão iguais**. Teoricamente, isso faz sentido.

Vamos levantar um ponto muito interessante. Pensemos em tipos mais simples: por exemplo, uma variável `a` igual a 4, um tipo inteiro, e uma variável `b` que é igual à variável `a`. No terminal, digitamos e rodamos os seguintes comandos:

```
$a = 4;
```

COPIAR CÓDIGO

```
$b = a;
```

COPIAR CÓDIGO

O PHP entende que o valor de `a` é `4`, **então** `b` **recebe 4,** ou seja, recebe o valor de `a`.

Quando executamos `echo $a`, o valor é `4`, e o mesmo ocorre com `echo $b`.

```
echo $a;
```

COPIAR CÓDIGO

```
4
```

```
echo $b;
```

COPIAR CÓDIGO

```
4
```

Se então usarmos `b++` para incrementar o valor de `b`, o resultado é que `b` se torna `5`

```
$b++;
```

COPIAR CÓDIGO

```
echo $b;
```

COPIAR CÓDIGO

Como retorno, obtemos o seguinte valor:

```
5
```

E `a` permanece `4` ao ser novamente exibido com `echo $a`.

```
echo $a;
```

COPIAR CÓDIGO

```
4
```

Isso é esperado.

Vamos observar uma situação semelhante, mas agora envolvendo variáveis relacionadas a filmes. Suponhamos que tenhamos essas duas variáveis, `filme` e `filme2`, ambas inicialmente com o **mesmo valor**. Inicialmente, definimos o nome do `filme` na variável filme como `'Filme'`.

```
$filme->nome = 'Filme';
```

COPIAR CÓDIGO

Em seguida, na variável `filme2`, atribuímos o nome `filme2`.

```
$filme2->nome = 'Filme 2';
```

COPIAR CÓDIGO

Ao usar `var_dump` para exibir o conteúdo da variável filme, esperamos visualizar apenas o nome filme, sem o número 2.

```
var_dump($filme);
```

COPIAR CÓDIGO

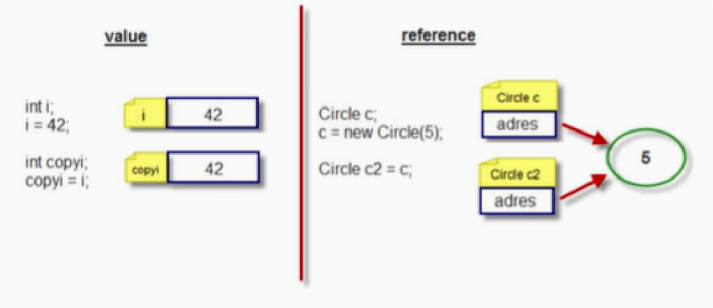
No entanto, ao executar `var_dump`, o resultado inesperado é "Filme 2".

```
object(Filme)#2 (2) {
["nome"]=>
string(7) "Filme 2"
["anoLancamento"]=>
int(2022)
["genero"]=>
uninitialized(string)
["nota"]=>
```

uninitialized(float) }

Quando lidamos com objetos, o valor das variáveis `filme` e `filme2` não é o próprio objeto, mas sim um **ponteiro** para o endereço de memória onde o objeto está armazenado.

Para compreender melhor essa distinção, vamos usar uma imagem que encontramos na *internet* para ilustrar a diferença entre **passar uma variável por valor e por referência**.



No contexto de objetos em PHP, lidamos com **referências**, enquanto em **variáveis** padrão, geralmente tratamos de **valores**.

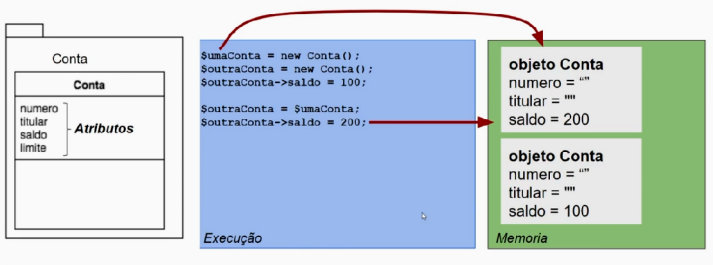
Isso significa que, na imagem que estamos analisando, em uma linguagem não especificada, temos dois lados representados. No lado esquerdo, que demonstra o uso de variáveis por **valor**, vemos uma variável inteira sendo definida com o valor `42`.

Embaixo, definimos uma variável inteira chamada de cópia (`copyi`) e atribuímos a ela o valor da variável inicial. Na prática, isso significa que teremos dois endereços de memória contendo o valor 42, pois a cópia é uma réplica exata desse valor.

Já quando lidamos com objetos, temos uma variável do tipo círculo, e estamos criando um objeto do tipo círculo e atribuindo essa variável `c` . A variável `c` vai possuir um endereço que **aponta para um espaço de memória que tem esse objeto** com todas as variáveis que estiverem lá dentro. E criamos uma segunda variável, `c2` , e dizemos que ela recebe o valor de `c` .

Na realidade, ao receber algo, ela não faz uma cópia de todos os valores, **mas sim do endereço em que estão armazenados**, ou seja, uma cópia do endereço. Isso significa que **ambas as variáveis apontam para o mesmo objeto**. Consequentemente, ao modificar o valor desse objeto através de qualquer uma das variáveis, o valor do objeto em si é alterado, já que ambas as variáveis estão referenciando esse mesmo objeto.

Para esclarecer isso melhor, vou usar uma apresentação de *slides* rápida.



Imagine que estamos construindo um **ambiente bancário**. Dentro desse contexto, temos uma conta específica.

Uma conta bancária possui diversos elementos como um número, um titular, um saldo, e outros valores associados. Para nomear corretamente esses elementos, utilizamos os termos **atributos** ou **propriedades**. Ambos os termos são adequados para se referir aos valores ou variáveis contidos dentro de uma classe e que são utilizados ao criar um objeto.

Vamos usar os termos corretos a partir de agora. Os **atributos** são **criados dentro de um objeto**. Quando definimos, por exemplo, a variável `umaConta` como `new Conta()`, **o PHP reserva um espaço de memória** e dentro desse espaço cria o objeto contendo o número, o titular e o saldo. Assim, essas informações ficam armazenadas no espaço de memória designado para o objeto.

Quando criamos outra conta usando a variável `outraConta = new Conta()`, o PHP reserva **outro espaço de memória**. Se tentarmos acessar e modificar, digamos, o

saldo dessa `outraConta`, o PHP simplesmente **localiza o endereço de memória correspondente e altera o valor do saldo** para algo como 100 reais, conforme o exemplo.

Quando atribuímos `outraConta` recebe `umaConta`, **ambas as variáveis passam a apontar para o mesmo objeto**. Isso significa que qualquer alteração feita em qualquer uma das variáveis afetará diretamente o objeto ao qual elas se referem.

Quando acessamos um objeto através de duas variáveis diferentes, qualquer modificação feita em uma delas refletirá no valor acessado pela outra variável. No exemplo dado, ambas as variáveis teriam o saldo de 200 após a modificação, **pois elas apontam para o mesmo objeto na memória**, compartilhando o mesmo endereço.

Compreender esse conceito é crucial, pois pode inicialmente causar confusão, mas é fundamental para entendermos. Por exemplo, ao passarmos um objeto do tipo filme como parâmetro para uma função, estamos lidando com o mesmo objeto originalmente criado, e quaisquer alterações feitas na função afetarão diretamente esse objeto.

Se essa função alterar algum valor do nosso objeto, quando a função terminar sua execução, ou seja, continuamos onde chamamos essa função, temos um **objeto alterado**. Precisamos ter essa consciência, precisamos ter isso em mente para saber que nossos objetos podem ser alterados, dependendo de como organizamos o nosso código.

Ao discutir a possibilidade de alteração, consideremos um cenário em que lançamos um filme. Temos uma variável chamada filme:

```
var_dump($filme);
```

COPIAR CÓDIGO

Ao executarmos, obtemos:

```
object(Filme)#2 (2) {

["nome"]=>

string(7) "Filme 2"

["anoLancamento"]=>

int(2022)

["genero"]=>
```

uninitialized(string)

["nota"]=>

uninitialized(float) }

Com o nome, `filme2`, e o ano de lançamento em 2022. Se essa variável for passada por parâmetro ou utilizada por outra pessoa desenvolvedora em uma rotina, e então decidimos modificar o ano de lançamento para 2023:

```
$filme->anoLancamento = 2023;
```

COPIAR CÓDIGO

Notamos que, após o lançamento do filme, estamos alterando o ano de lançamento. Isso não faz sentido. Ao realizar um `var_dump`, veremos que o ano de lançamento foi modificado sem problemas.

```
var_dump($filme);
```

COPIAR CÓDIGO

object(Filme)#2 (2) {

["nome"]=>

```
string(7) "Filme 2"

["anoLancamento"]=>

int(2023)

["genero"]=>

uninitialized(string)

["nota"]=>

uninitialized(float)

}
```

Nesse caso, ainda mantemos nossos dados e comportamentos separados, permitindo a **manipulação livre de todos os dados de qualquer lugar**. Por exemplo, podemos alterar a nota de um filme em qualquer parte do código, colocando a nota desejada, ou modificar o gênero do filme sem restrições.

Conclusão e Próximos Passos

No mundo real, certas coisas não mudam após o lançamento de um filme: o gênero, o ano de lançamento e, possivelmente, o nome permanecem estáveis. Enquanto a nota pode variar, outros elementos como o

ano de lançamento e o gênero são fixos. Isso aponta para a necessidade de um **controle mais eficiente no acesso aos nossos dados**.

Na próxima aula, abordaremos **como definir comportamentos para um filme e como controlar o acesso aos valores dentro dele**. Aguardo vocês para tornar nosso modelo de filme mais robusto!